However, such usage is *not recommended*, because it can lead to sloppy thinking and bad design. In particular, the class invariants are typically broken while the lock is held, which means that the second member function needs to work even when called with the invariants broken. It's usually better to extract a new private member function that's called from both member functions, which does not lock the mutex (it expects it to already be locked). You can then think carefully about the circumstances under which that new function can be called and the state of the data under those circumstances.

3.4. Summary

In this chapter I discussed how problematic race conditions can be disastrous when sharing data between threads and how to use `std::mutex` and careful interface design to avoid them. You saw that mutexes aren't a panacea and do have their own problems in the form of deadlock, though the C++ Standard Library provides a tool to help avoid that in the form of `std::lock()`. You then looked at some further techniques for avoiding deadlock, followed by a brief look at transferring lock ownership and issues surrounding choosing the appropriate granularity for locking. Finally, I covered the alternative data-protection facilities provided for specific scenarios, such as `std::call_once()`, and `boost::shared_mutex`.

One thing that I haven't covered yet, however, is waiting for input from other threads. Our thread-safe stack just throws an exception if the stack is empty, so if one thread wanted to wait for another thread to push a value on the stack (which is, after all, one of the primary uses for a thread-safe stack), it would have to repeatedly try to pop a value, retrying if an exception gets thrown. This consumes valuable processing time in performing the check, without actually making any progress; indeed, the constant checking might *hamper* progress by preventing the other threads in the system from running. What's needed is some way for a thread to wait for another thread to complete a task without consuming CPU time in the process. Chapter 4 builds on the facilities I've discussed for protecting shared data and introduces the various mechanisms for synchronizing